Feature Engineering

Data Science

Feature Engineering

- missing values
- numerical features: scaling, transformations
- categorical features: encoding
- feature selection
- data leakage
- imbalanced data

Why Feature Engineering Matters

idea: transforming raw data into meaningful inputs (features)

→ creating new variables from existing data

raw data:

- messy, incomplete, hard to interpret
- example: timestamps, IDs, text blobs

→ good features make patterns learnable

engineered features:

- structured, meaningful, model-ready
- example: hour of day, purchase frequency, text sentiment

Demand Forecasting as Example for Feature Engineering

seasonality

day of week, payday, week of year

weather

event-like: first sunny weekend, snowstorm, …

events (e.g., holidays)

time window around event (e.g., monotonic)

promotion

days since start of promotion period

price

ratio of reduced and normal price, price-demand elasticity

product information

text (name/description) or image embeddings

Features & Model Choice

- garbage in → garbage out
- better features → simpler models can perform well
- poor features → even complex models struggle

common misconception: better model = better results

but feature quality often matters more than algorithm choice

→ a simple model + great features can outperform a complex model + poor features

Model Pipeline

extract features

- help the ML algorithm to better understand the data
- impose assumptions hard to discover in the raw data

choose ML algorithm

- from open-source libraries like scikit-learn or pytorch, rarely write an own one
- many different algorithms available, differently suited for given task

hyperparameter tuning

- variety of different forms
- model settings not all automatically adjusted by the ML algorithm

Before: Preprocessing Pipeline

typical steps:

1. handle missing values
2. fix inconsistencies
3. treat outliers
4. scale and transform features (slide before: extract features)

understanding your data is the first step in feature engineering
→ visualization

Handling Missing Values

common issue: incomplete observations

examples: missing age, unknown category, gaps in time series

challenges: bias if handled poorly, loss of information if removed

common strategies:

- remove rows (if few and random)
- remove columns (if mostly missing)
- imputation (fill in values)

Imputation Techniques

- mean / median imputation (numerical data)
- mode imputation (categorical data)
- forward/backward fill (time series)
- model-based imputation

→ choice depends on data and assumptions

sometimes missingness itself is useful (informative):

- missing income → may indicate unemployment
- missing responses → may indicate behavior

→ create an additional “missing indicator” feature

Noise & Outliers

noisy data: errors, inconsistencies, or randomness

- examples: incorrect entries, sensor errors, typos
- reduce model performance, can mislead feature engineering

outliers: data points significantly different from others

- examples: extreme or impossible values (e.g., negative age)
- detection: statistical rules or visualization (boxplots, histograms)
- options for handling: remove, clip, transform, keep (if meaningful)

Main Data Types

for homogeneous data (text, images)
→ feature learning (deep learning)

tabular data (heterogeneous):

data type	**key challenge**
numerical	scale, outliers, skew
categorical	encoding, high cardinality
ordinal	preserving order
time series	temporal structure
text	unstructured complexity

→ each requires different handling

Numerical Data

continuous or discrete numbers

examples:
- age, income, temperature
- number of purchases

challenges:
- different scales (e.g., income vs. age)
- outliers
- skewed distributions

Feature Scaling

some models depend on distance:

- kNN
- SVM
- gradient-based methods (e.g., neural networks)

→ features on different scales can distort learning

Normalization & Standardization

normalization (Min-Max scaling): scales values to $[0, 1]$

standardization (Z-score): centers data around mean $= 0$ (with standard deviation $= 1$)

$$z = \frac{x - \mu}{\sigma}$$

- use normalization when bounds matter (e.g., neural networks)
- use standardization when data is roughly Gaussian (default choice in many cases)

Scaling vs. Transformation

scaling: adjusts range (e.g., normalization)

transformation: changes shape (e.g., $\log$)

→ often used together

Feature Transformation

- changing the representation of existing features
- making patterns easier for models to learn

→ same data, better representation

when to transform:

- distribution is highly skewed
- outliers dominate
- model assumptions (e.g., linearity) are violated

Log Transformation

used for right-skewed data (e.g., income, prices)

effect: compresses large values, expands small values

example: income

- before: $1\text{k}, 2\text{k}, 5\text{k}, 100\text{k}$
- after $\log$: values become closer in scale

→ makes distributions more symmetric, easier for models to detect patterns

alternatives:

- square root transformation: less aggressive than $\log$, useful for moderate skew → keeps more relative differences than log transform
- Box-Cox transformation: family of transformations, finds optimal parameter automatically → makes data closer to normal distribution

Binning (Discretization)

convert continuous values into categories
example: age → $\{0–18, 19–35, 36–60, 60+\}$

options: equidistant or equistatistics (or equal-frequency) binning

- simplifies relationships
- reduces noise (lower variance)
- helps certain models
- but may lose information (higher bias)

Polynomial and Interaction Features

create new features using powers (e.g., x^2, x^3)
→ allows linear models to capture nonlinear relationships

combine multiple features (e.g., $x_1 \cdot x_2$)
→ captures relationships between variables
example: $\mathrm{Age} \times \mathrm{Income}$

special type for categorical data: feature crosses
example: $\mathrm{City} \times \mathrm{Device\ Type}$

Feature Creation

building new features from existing ones

- adding information that was not explicitly present
- captures hidden relationships

→ domain knowledge

often leads to the biggest performance gains
but always check performance and generalization: not all features help

many possible forms …

Aggregation Features

combine data across groups

examples:

- average purchase per user
- total transactions per day
- max/min values per group

→ often created using group operations

aggregation example: transactions per user
new features: total spend per user, average order value, purchase frequency
→ summarizes behavior effectively

More Examples

- ratio features: e.g., price per square meter → often more informative than raw values
- difference features: e.g., current price − previous price → useful for trend and behavior analysis
- feature extraction from timestamps: e.g., day of week → turns raw time into meaningful patterns
- cyclical features: e.g., hour of day → encode using sine/cosine transformations
- count-based features: e.g., number of logins → captures activity level
- flag/indicator features: e.g., is premium user → simple but often powerful

Categorical Data

values representing categories or groups

examples: country, color, product type

problem: ML models work with numbers, not text

→ cannot be directly used by most models

→ convert categories into numerical representations

Encoding

defines how categorical information is represented numerically

different possibilities:

- label encoding & ordinal encoding
- one-hot encoding
- target encoding & leave-one-out encoding
- embeddings

Label & Ordinal Encoding

both methods assign an integer to each category

label encoding:

- used for unordered categories (especially target variables)
- example: "Red", "Green", "Blue" → arbitrary integers

ordinal encoding:

- used for categories with a natural order
- example: "Low" → 1, "Medium" → 2, "High" → 3
- not suitable for unordered categories (introduces false ordering)

alternative for high-cardinality features: binary encoding (convert categories into binary code)

One-Hot Encoding

creates binary columns for each category:

Color	Red	Blue	Green
Red	1	0	0

no artificial ordering, works well for most models

but increases dimensionality → high-cardinality problem

example: for categorical variable “User ID” with 100,000 unique values, one-hot encoding creates 100,000 columns (sparse data)

→ overfitting risk (curse of dimensionality), very inefficient

Target & Leave-One-Out Encoding

encodes categories using target statistics (e.g., average target value)

example: predict individual houses prices in 100,000 €

City	**Average Price**
A	200
B	150

- handles high-cardinality variables well
- often results in very powerful features

but high risk of overfitting (target leakage)

→ leave-one-out encoding: exclude current row when calculating target statistics

Frequency/Count Encoding

replace category with frequency (count) or relative frequency

example:

category appears 500 times → value = 500

Text Features

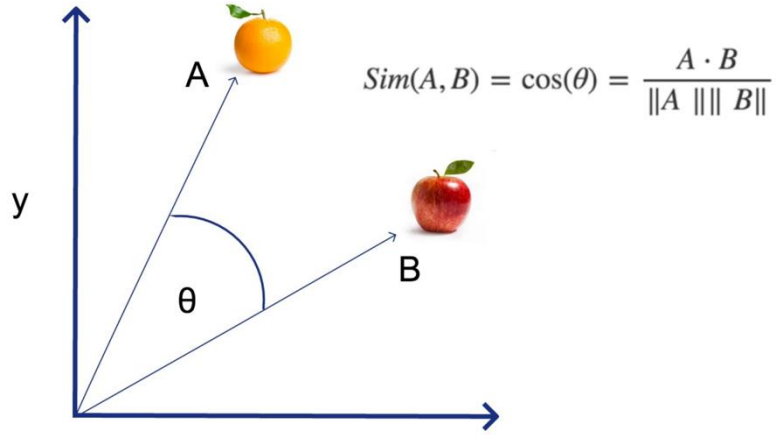
different methods to deal with variables containing full text:

- Bag-of-Words (BoW): count word occurrences
  - example: “I love data” → {I:1, love:1, data:1}
  - limitations: ignores word order, high-dimensional, sparse representation
- Term Frequency–Inverse Document Frequency (TF-IDF): weighs informative (rare) words higher, downweights common words
- embeddings (contextual embeddings via transformer architecture)

Embeddings

representation of entities by vectors

→ semantic similarity



most famous application: word embeddings

→ associations (natural language processing)

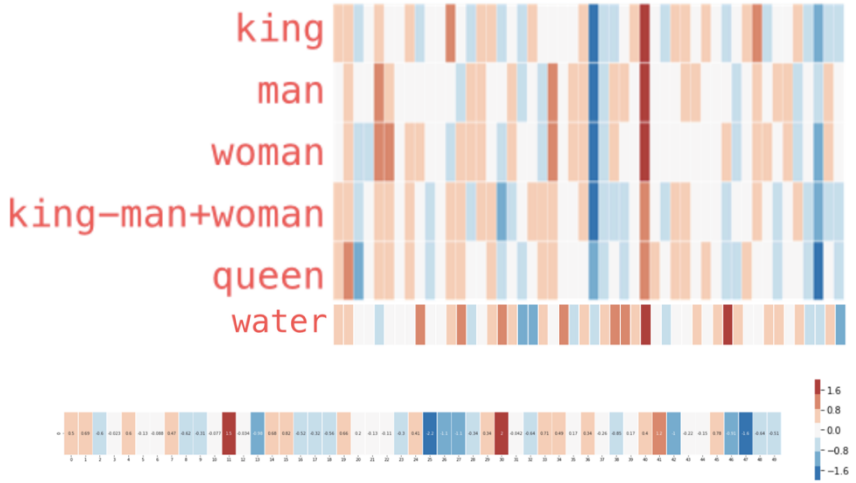
but general concept: embeddings of (categorical) features (e.g., products in recommendation engines)

learned via co-occurrence (e.g., word2vec)

end-to-end learning: embeddings layer in deep neural network, e.g., multi-layer perceptron (MLP)

but also direction of difference vectors interesting (analogies):

king − man + woman ~= queen



source

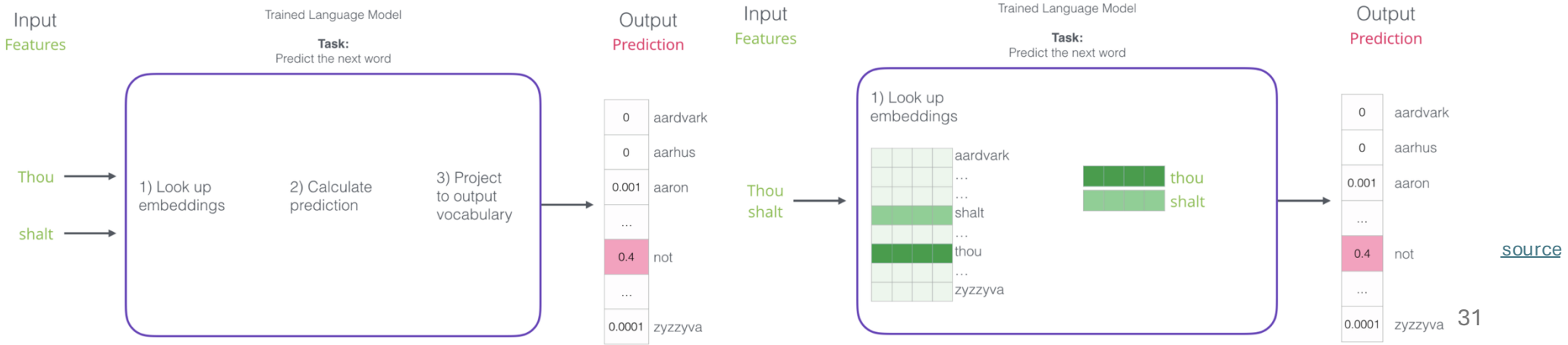
Word Embeddings as Part of Language Model

language models contain embedding matrix as part of learned parameters

typically several hundred dimensions for word vectors (to be compared with vocabulary sizes of many thousands)

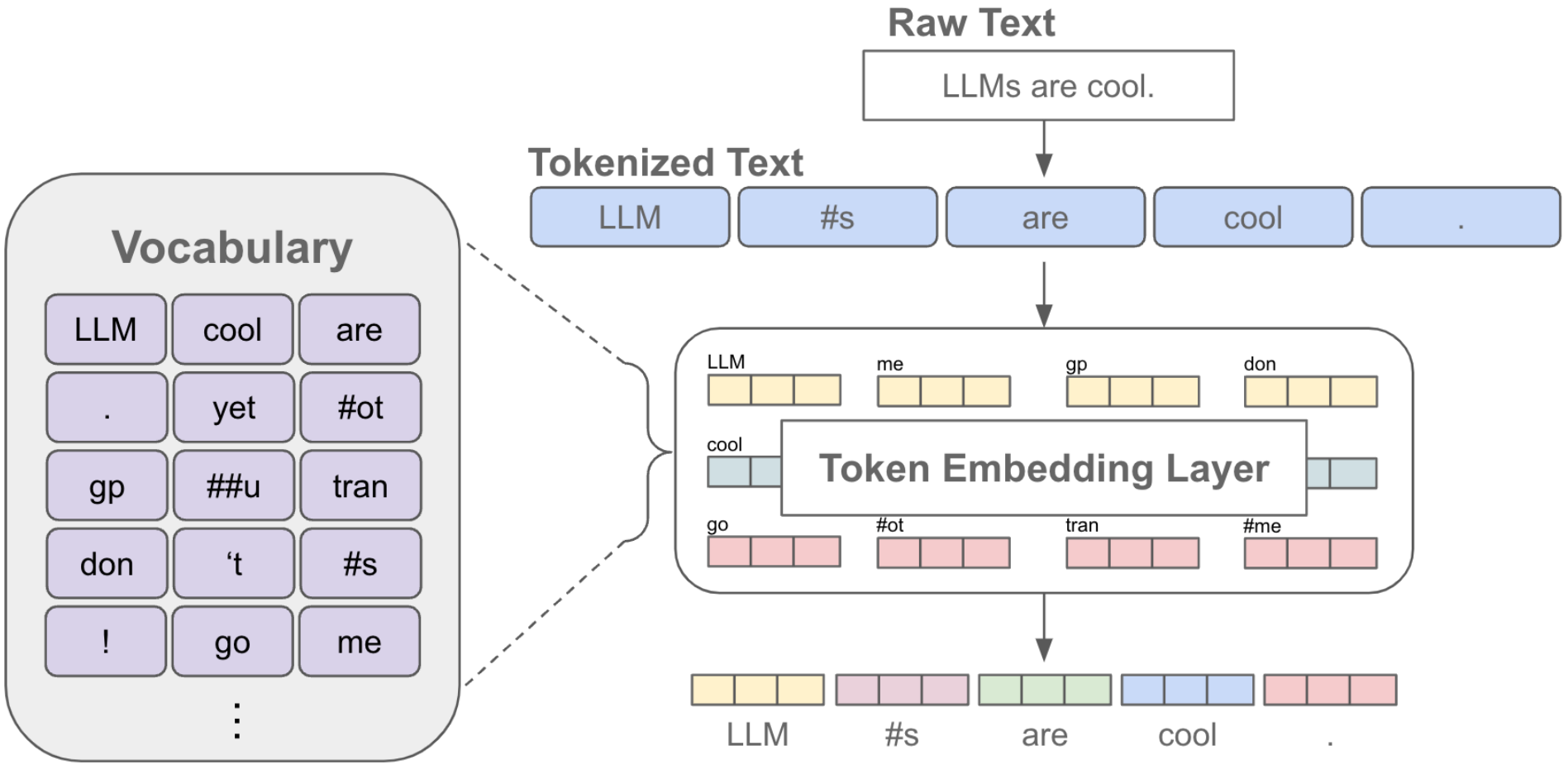
(can also be extracted and subsequently used as pretrained embeddings for other task)

next-word prediction:



source

Tokenization & Embeddings



source

Other Complex Data Types

image data

- pixel intensities as input
- feature extraction mainly via deep learning (→ computer vision)

graph data

- nodes and edges → relationships matter
- examples: social networks, recommendation systems
- modern best practice: graph embeddings / Graph Neural Networks

Feature Selection

too many features: curse of dimensionality

→ choose a subset of relevant features, remove irrelevant or redundant ones (keep signal, remove noise)

- improves model performance
- reduces overfitting
- speeds up training
- enhances interpretability

Feature Selection Methods

filter methods: evaluate features independently of the model

- remove features highly correlated with each other
- keep those strongly correlated with target

embedded methods: perform feature selection during model training

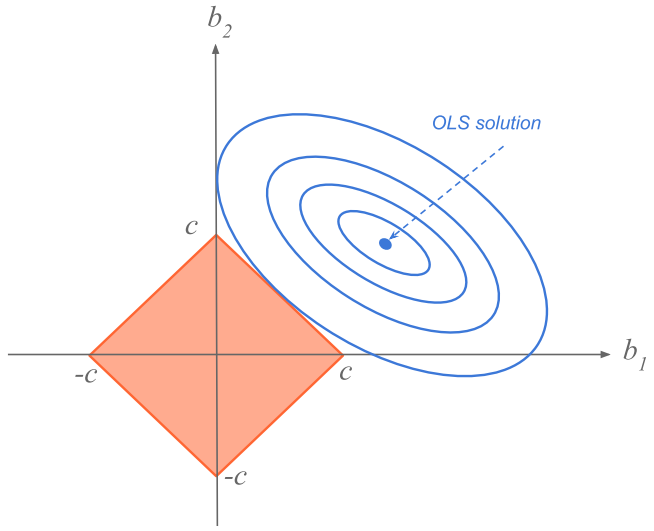
- tree-based models: rank features by importance
- Lasso regression

Lasso Regression

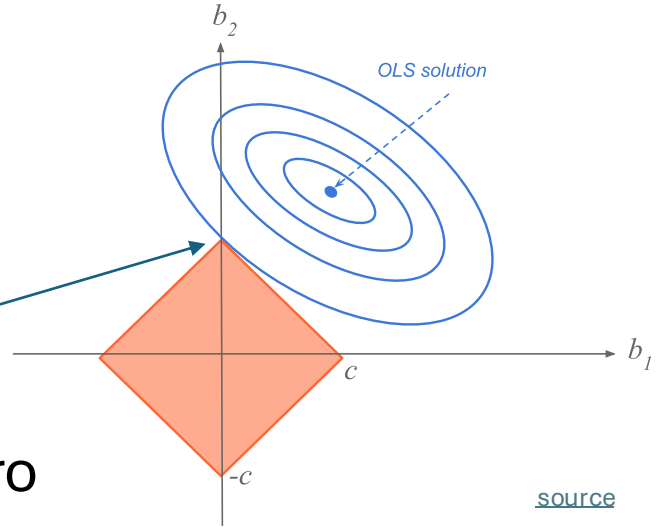
Least **A**bsolute **S**hrinkage and **S**election **O**perator

adding **penalty term** on parameter L^1 norm to cost function

$$\tilde{J}(\hat{\boldsymbol{\theta}}) = J(\hat{\boldsymbol{\theta}}) + \lambda \cdot \sum_j |\hat{\theta}_j|$$



corresponds to imposing constraint on parameters to lie in L^1 region (size controlled by λ)



drives some coefficients to zero

→ automatic feature selection

source

Related Concept: Dimensionality Reduction

high dimensionality: large number of features

instead of selecting features transform them into fewer dimensions

example: Principal Component Analysis (PCA)

→ reduces dimensionality but loses interpretability

Data Leakage

definition:

using information not available at prediction time

example:

using future data to create features

→ leads to unrealistically high performance

Train/Test Leakage

common mistake:

compute statistics (mean, std) on full dataset

correct approach:

fit preprocessing on training data only, apply to test data

Feature Drift

feature distributions change over time

examples:

- user behavior shifts
- market conditions change

→ model performance degrades in production

even more critical: target drift

Imbalanced Data

imbalanced: one class significantly outnumbers others

example fraud detection:

- 99% non-fraud
- 1% fraud

→ accuracy is misleading
(use precision, recall, $\mathrm{F1}$, …)

models tend to favor the majority class and ignore minority class

→ leads to misleading performance

Options for Handling Imbalanced Data

adjust the dataset by resampling

- oversampling: duplicate or generate minority samples → keeps all data, but risk of overfitting
- undersampling: remove majority samples → simpler dataset, but loss of information

apply class weights: assign higher penalty to minority class (many models support this directly, e.g., in scikit-learn)

adjust decision threshold (e.g., lower threshold → higher recall)

create features that better separate minority class or highlight rare patterns

Pipelines

use pipelines to:

- ensure reproducibility
- avoid leakage
- simplify workflows

→ automate preprocessing

example tool: scikit-learn pipelines